

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

ФАКУЛЬТЕТ ПРИКЛАДНОЙ МАТЕМАТИКИ И ИНФОРМАТИКИ

Кафедра математического моделирования и анализа данных

Сазончик Иван Михайлович

**ОЦЕНКА КАЧЕСТВА ЭНТРОПИИ В
БИОЛОГИЧЕСКИХ ГЕНЕРАТОРАХ
СЛУЧАЙНЫХ ЧИСЕЛ НА ОСНОВЕ
ПОВЕДЕНЧЕСКИХ ФАКТОРОВ**

Курсовая работа

студента 4 курса 9 группы

Научный руководитель:

старший преподаватель

М.А. Казловский

Минск, 2025

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ	2
1 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ БИОЛОГИЧЕСКИХ ГЕНЕРАТОРОВ СЛУЧАЙНЫХ ЧИСЕЛ	5
1.1 Классификация источников энтропии	5
1.2 Особенности источников, основанных на активности оператора	6
2 РАЗРАБОТКА ПРОГРАММНОГО КОМПЛЕКСА ДЛЯ СБОРА ЭНТРОПИИ ДВИЖЕНИЙ ГЛАЗ	9
2.1 Общий вид интерфейса программного комплекса	9
2.2 Архитектура системы	10
2.3 Алгоритмы трекинга	13
2.4 Алгоритм извлечения энтропии	17
2.5 Реализация	20
3 МЕТОДОЛОГИЯ ОЦЕНКИ КАЧЕСТВА ЭНТРОПИИ	23
3.1 Тестовый стенд	23
3.2 Инструменты оценки	24
3.3 Критерии успешности	25
4 ЭКСПЕРИМЕНТАЛЬНЫЕ РЕЗУЛЬТАТЫ И ИХ АНАЛИЗ	26
4.1 Результаты тестирования	26
4.2 Анализ результатов по сценариям	27
4.3 Ограничения и перспективы	28
ЗАКЛЮЧЕНИЕ	29
СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ	31
ПРИЛОЖЕНИЕ А	32

ВВЕДЕНИЕ

В современных условиях цифровой трансформации и повсеместного внедрения криптографических систем надёжная генерация случайных чисел является фундаментальной задачей информационной безопасности. Случайные числа и ключевые последовательности используются в широком спектре приложений: от шифрования данных и цифровых подписей до протоколов аутентификации и лотерейных систем. Качество таких последовательностей напрямую определяет устойчивость криптосистем к атакам.

Традиционные генераторы случайных чисел (ГСЧ) можно разделить на три основных класса: алгоритмические (псевдослучайные), физические (аппаратные) и комбинированные. Алгоритмические ГСЧ, хотя и обладают высокой скоростью и воспроизводимостью, по своей природе детерминированы и уязвимы при компрометации начального состояния (*seed*). Физические ГСЧ, основанные на измерениях шумовых процессов (тепловой шум, радиоактивный распад и др.), обеспечивают истинную случайность, но часто требуют специализированного и дорогостоящего оборудования, а их выход может быть подвержен нестабильности и внешним помехам. Всё это актуализирует поиск новых, доступных, недетерминированных и устойчивых источников энтропии, интегрируемых в повседневные вычислительные среды.

Перспективным направлением в этой области является использование биометрических данных человека. Биометрия, изучающая уникальные физиологические и поведенческие характеристики индивидов, предлагает два основных типа источников энтропии. К *физиологическим* относятся относительно стабильные во времени, но сложные для непрерывного измерения параметры, такие как ритмы сердца (ЭКГ), мозговые волны (ЭЭГ), характеристика кровотока. *Поведенческие* факторы, напротив, отражают динамические и зачастую непредсказуемые паттерны деятельности человека: динамика нажатий клавиш, движения компьютерной мыши, жесты, речь и, что особенно интересно, саккадические и прослеживающие движения глаз.

Движения глаз представляют собой непрерывный, высокочастотный и в значительной степени произвольный процесс, управляемый сложным взаимодействием когнитивных, перцептивных и нейромоторных систем. Траектория взгляда даже при фиксации на статичном объекте демонстрирует микродвиже-

ния (тремор, дрейф, микросаккады), которые являются принципиально непредсказуемыми на длительных интервалах и потенциально богатым источником энтропии. Использование этого биометрического канала не требует инвазивного или дорогого оборудования — достаточно стандартной веб-камеры и алгоритмов компьютерного зрения, что открывает возможности для создания программных биологических ГСЧ (BioRNG) массового применения.

Целью данной работы является разработка, программная реализация и комплексная оценка качества биологического генератора случайных чисел, основанного на анализе поведенческого фактора — траектории движения глаз человека.

Для достижения поставленной цели в ходе исследования были последовательно решены следующие **задачи**:

1. Разработка программного комплекса для захвата видеопотока с веб-камеры, трекинга лицевых ориентиров и выделения параметров движения глаз (координаты взгляда, скорость, направление, состояние век) в реальном времени.
2. Реализация алгоритма извлечения и накопления энтропии из потока поведенческих данных, включающего этапы фильтрации (исключение кадров с закрытыми глазами, подавление шума), квантования извлечённых параметров и формирования битовой последовательности.
3. Оценка статистического качества и непредсказуемости генерируемых битовых последовательностей с использованием набора стандартизированных тестов NIST SP 800-90B, предназначенных для оценки источников энтропии.
4. Экспериментальное исследование влияния контекста деятельности пользователя (произвольные движения глаз, серфинг в интернете, просмотр видео) на количественные и качественные характеристики извлекаемой энтропии.

Объектом исследования выступает энтропия, извлекаемая из траектории движения глаз человека как динамического поведенческого биометрического сигнала.

Предметом исследования являются алгоритмы и методы извлечения, обработки и статистической оценки энтропии из данных трекинга взгляда.

В работе применяется комплекс **методов исследования**:

- Методы *компьютерного зрения* и машинного обучения (библиотеки MediaPipe, OpenCV) для детекции лица, локализации ключевых точек глаз и

отслеживания их перемещений.

- Методы *цифровой обработки сигналов и статистического анализа* для фильтрации данных, выделения признаков и первичной оценки их стохастических свойств.
- Стандартизированные методы *тестирования энтропии* (NIST SP 800-90B), включая оценку минимальной энтропии, тесты на независимость и идентичность распределения (IID), а также тесты для не-IID данных.
- Метод *экспериментального исследования* для сбора данных в различных сценариях и сравнительного анализа результатов.

Практическая значимость работы заключается в демонстрации возможности построения доступного программного ГСЧ с истинной энтропией, использующего стандартные аппаратные средства. Теоретическая ценность состоит в исследовании свойств энтропии, порождаемой сложной биологической системой (окулоmotorной), и влияния на эти свойства внешнего когнитивного контекста.

Глава 1

ТЕОРЕТИЧЕСКИЕ ОСНОВЫ БИОЛОГИЧЕСКИХ ГЕНЕРАТОРОВ СЛУЧАЙНЫХ ЧИСЕЛ

1.1 Классификация источников энтропии

В соответствии со стандартом СТБ 34.101.27-2022 «Средства криптографической защиты информации. Требования безопасности» генераторы случайных чисел (ГСЧ), используемые в средствах криптографической защиты информации (СКЗИ), классифицируются по типам источников случайности (энтропии), на основе которых формируются непредсказуемые последовательности. Стандартом выделяются следующие категории источников случайности:

Физические источники случайности – основаны на измерении стохастических физических процессов в аппаратных устройствах. К ним относятся:

- шумовые процессы в радиоэлектронных и оптических компонентах (тепловой шум, дробовой шум);
- квантовые явления (статистика фотонов, квантовая запутанность);
- радиоактивный распад;
- атмосферные помехи.

Согласно п. 5.10 стандарта, физические источники являются предпочтительными для использования в СКЗИ. Они обеспечивают истинную случайность (недетерминированность), однако требуют наличия специализированных аппаратных компонентов, могут быть чувствительны к изменениям внешних условий (температура, напряжение), а их производительность ограничена скоростью физических процессов.

Системные источники случайности – используют состояния, процессы и события системной среды, в которой функционирует СКЗИ. Примеры таких источников:

- показания системных таймеров и счётчиков;
- сетевая активность (время прибытия пакетов);
- прерывания и планирование процессов операционной системы.

Системные источники широко доступны в программных СКЗИ, но их энтропия может быть ограничена предсказуемостью системных событий и потенциальной возможностью влияния на них злоумышленника.

Источники, основанные на активности оператора — используют поведенческие паттерны человека при взаимодействии с вычислительной системой. К ним относятся:

- динамика работы с клавиатурой (временные интервалы между нажатиями);
- движения манипулятора «мышь» (траектория, скорость, ускорение);
- окулomotorная активность (движения глаз, саккады, фиксации взгляда).

Данные источники могут быть реализованы с использованием стандартных периферийных устройств (веб-камер, клавиатур, мышей), что делает их доступными для массового применения. Однако их энтропия зависит от психофизиологического состояния оператора, требует фильтрации артефактов и может поднимать вопросы конфиденциальности биометрических данных.

1.2 Особенности источников, основанных на активности оператора

Поведенческие источники энтропии, в отличие от физиологических, характеризуются высокой динамичностью и непосредственной связью с когнитивными процессами. Это определяет их специфические особенности, которые необходимо учитывать при разработке биологических ГСЧ.

Непредсказуемость микро-движений. Даже при выполнении, казалось бы, простых и повторяемых задач, поведенческие паттерны человека демонстрируют существенную вариабельность на микроуровне. Например:

- При фиксации взгляда на неподвижном объекте глаза совершают произвольные микродвижения трех типов: *тремор* (высокочастотные колебания с амплитудой 20-40 угловых секунд), *дрейф* (медленные смещения) и *микросаккады* (быстрые корректирующие движения)
- При печати на клавиатуре интервалы между нажатиями одних и тех же клавиш комбинаций варьируются в пределах 20-100 мс даже у опытных машинисток
- Траектория движения мыши при выполнении однотипных задач (например, клик по кнопке) никогда не повторяется точно из-за мышечной усталости,

дрожания руки и неосознанных корректировок

Эта микро-вариабельность обусловлена сложностью нейромоторной системы человека, которая включает множество источников шума: стохастичность высвобождения нейромедиаторов, вариабельность проведения нервных импульсов, нелинейность мышечного ответа. Эти физиологические шумы делают поведенческие паттерны принципиально непредсказуемыми на достаточно длительных интервалах.

Влияние когнитивных процессов. Поведенческие паттерны тесно связаны с текущим когнитивным состоянием человека, что создает как возможности, так и вызовы для генерации энтропии:

- *Внимание и концентрация* – уровень внимания влияет на стабильность движений. При высокой концентрации движения становятся более точными, но при этом могут уменьшаться случайные колебания, что потенциально снижает энтропию
- *Усталость и стресс* – физическая и умственная усталость увеличивает вариабельность движений, но также может вносить систематические смещения (например, замедление реакции)
- *Эмоциональное состояние* – эмоции влияют на двигательную активность: тревожность может увеличивать частоту микродвижений, тогда как спокойное состояние – уменьшать
- *Когнитивная нагрузка* – выполнение параллельных задач (мультизадачность) существенно изменяет поведенческие паттерны, часто увеличивая их непредсказуемость

Контекстуальная зависимость. Качество и количество энтропии, извлекаемой из поведенческих источников, существенно зависит от контекста деятельности:

- *Произвольная активность* – когда пользователь сознательно пытается генерировать «случайные» движения, возникает парадоксальный эффект: сознательный контроль снижает истинную случайность, так как человек склонен к созданию паттернов и избеганию «крайних» значений
- *Естественная деятельность* – при выполнении осмысленных задач (чтение, серфинг в интернете, просмотр видео) движения носят более естественный характер и могут предоставлять более качественную энтропию, так как когнитивные процессы фокусируются на содержании задачи, а не на контроле

движений

- *Тип задачи* – разные виды деятельности генерируют энтропию с различными характеристиками. Например, чтение текста порождает регулярные саккадические движения с фиксациями, тогда как свободное рассматривание изображений создает более хаотичную траекторию взгляда

Технические особенности измерения. Поведенческие источники требуют специальных подходов к измерению и обработке сигналов:

- *Дискретизация и квантование* – аналоговые поведенческие сигналы (координаты взгляда, временные метки) должны быть преобразованы в цифровую форму, что вносит дополнительный шум квантования
- *Артефакты измерений* – аппаратные ограничения (частота кадров камеры, разрешение сенсоров) и внешние факторы (освещение, помехи) могут искажать измеряемые параметры
- *Предобработка сигналов* – необходима фильтрация высокочастотного шума, устранение выбросов и компенсация систематических ошибок измерений

Поведенческие источники энтропии, и особенно движения глаз, представляют особый интерес для создания биологических ГСЧ. Окуломоторная система сочетает в себе высокую частоту обновления (до 1000 Гц в специализированных системах), богатство непредсказуемых микро-движений и относительно простую техническую реализацию измерения с использованием обычных веб-камер. Эти особенности делают движения глаз перспективным кандидатом для создания доступных, но криптографически стойких источников случайности.

Глава 2

РАЗРАБОТКА ПРОГРАММНОГО КОМПЛЕКСА ДЛЯ СБОРА ЭНТРОПИИ ДВИЖЕНИЙ ГЛАЗ

2.1 Общий вид интерфейса программного комплекса

На рисунке 2.1 представлен общий вид пользовательского интерфейса разработанного программного комплекса для сбора энтропии движений глаз.

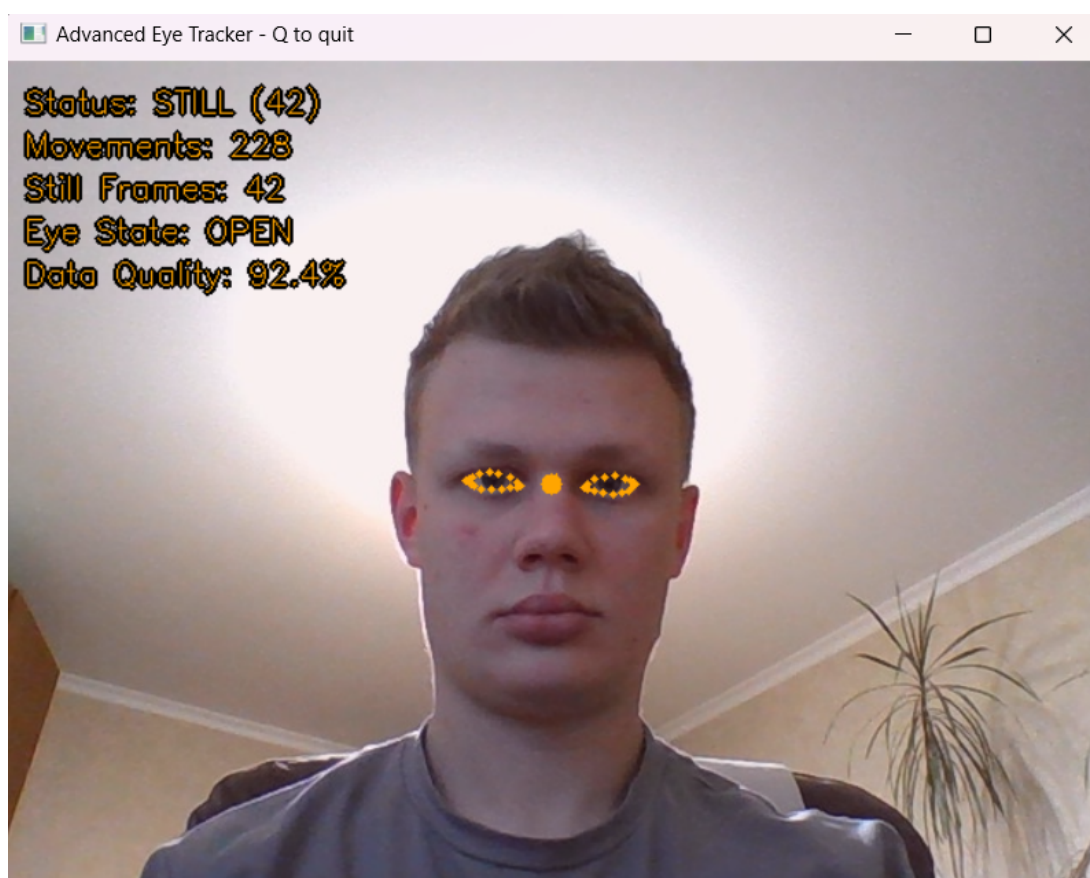


Рисунок 2.1 — Общий вид интерфейса программного комплекса

Интерфейс программного комплекса реализован в виде окна видеопотока с веб-камеры, на которое в реальном времени накладываются элементы визуализации работы системы. В центральной части окна отображается изображение пользователя, получаемое с камеры, а также ключевые точки области глаз, используемые для анализа направления взгляда.

В процессе работы системы на изображении визуализируется текущая точка взгляда, а также отображается информация о наличии или отсутствии движения

глаз. При обнаружении значимого движения система фиксирует соответствующее событие и использует его параметры для извлечения энтропии.

В левой верхней части интерфейса выводится служебная информация, включающая текущий статус системы, количество зафиксированных движений, число статичных кадров подряд, состояние глаз (открыты или закрыты), а также оценка качества собранных данных. Данная информация позволяет оператору в реальном времени контролировать корректность работы алгоритмов и качество входных данных.

Разработанный интерфейс ориентирован на минимальное вмешательство пользователя в процесс работы системы и предназначен преимущественно для визуального контроля и отладки. Основной процесс сбора энтропии и генерации случайных битов происходит автоматически и не требует активных действий со стороны пользователя.

2.2 Архитектура системы

Программный комплекс для сбора энтропии движений глаз был разработан с использованием модульного подхода, что обеспечивает гибкость, масштабируемость и возможность независимой модификации отдельных компонентов. Архитектура системы представлена на рисунке 2.2.

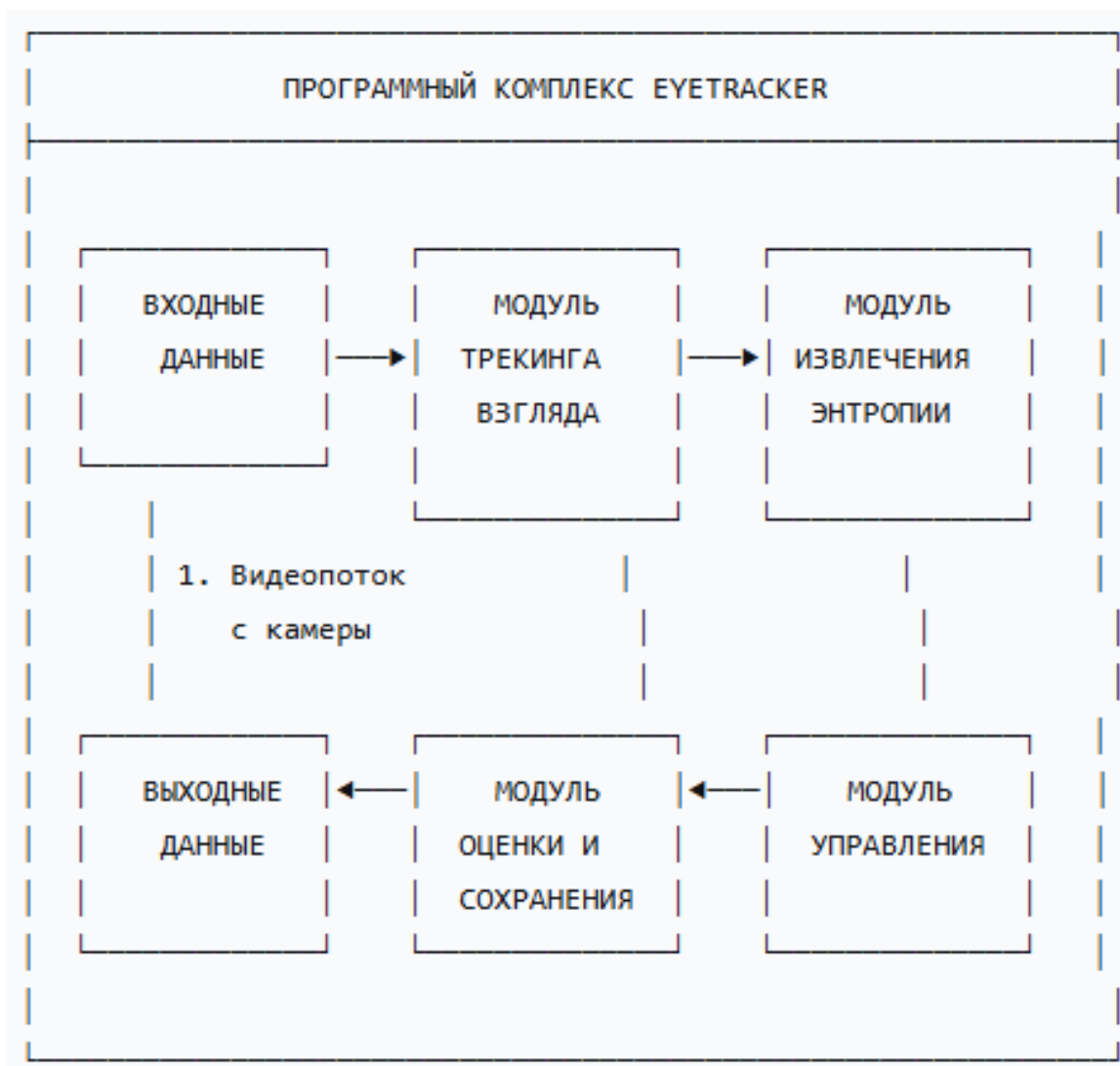


Рисунок 2.2 — Архитектура программного комплекса для сбора энтропии движений глаз

Модуль трекинга взгляда (AdvancedGazeTracker) является ядром системы и выполняет следующие функции:

- Захват видеопотока с веб-камеры в реальном времени с частотой до 30 кадров в секунду
- Детекция лица и локализация 468 лицевых ориентиров с использованием нейросетевой модели MediaPipe Face Mesh
- Выделение 16 ключевых точек для каждого глаза с последующим вычислением центра глаза
- Определение состояния век (открыты/закрыты) на основе отношения вертикального и горизонтального расстояний между ключевыми точками
- Расчет координат точки взгляда на экране с комбинированием методов цен-

тров глаз и смещения зрачков

- Детекция значимых движений глаз с применением пороговой фильтрации по величине и временным интервалам
- Визуализация результатов трекинга с наложением графических элементов на видеопоток

Модуль извлечения энтропии (MovementBasedCryptoGenerator) реализует криптографическую обработку данных:

- Прием потока данных о движениях глаз от модуля трекинга
- Извлечение энтропии из 6 независимых источников: координат X и Y, величины движения, направления, временных меток и физиологических параметров
- Квантование аналоговых значений в битовые последовательности с использованием младших битов для увеличения случайности
- Формирование пула энтропии по принципу FIFO (First-In-First-Out)
- Генерация криптографических ключей заданной длины (128 или 256 бит)

Модуль оценки и сохранения данных обеспечивает персистентность и анализ:

- Сохранение сырых битовых последовательностей в бинарные файлы для последующего статистического анализа
- Реализация интерактивного интерфейса управления с обработкой клавиатурных команд
- Расчет статистических показателей в реальном времени: количество движений, процент движения, качество данных
- Экспорт сгенерированных ключей в файлы формата .bin для использования в криптографических приложениях

Взаимодействие между модулями осуществляется через четко определенные программные интерфейсы. Модуль трекинга передает объекты данных, содержащие координаты взгляда, параметры движения и метаданные, в модуль извлечения энтропии. Последний аккумулирует данные до достижения порогового значения, после чего инициирует процедуру генерации ключа. Модуль оценки работает асинхронно, сохраняя данные на диск и обновляя статистику без блокировки основных процессов.

2.3 Алгоритмы трекинга

Использование MediaPipe Face Mesh для детекции лицевых ориентиров

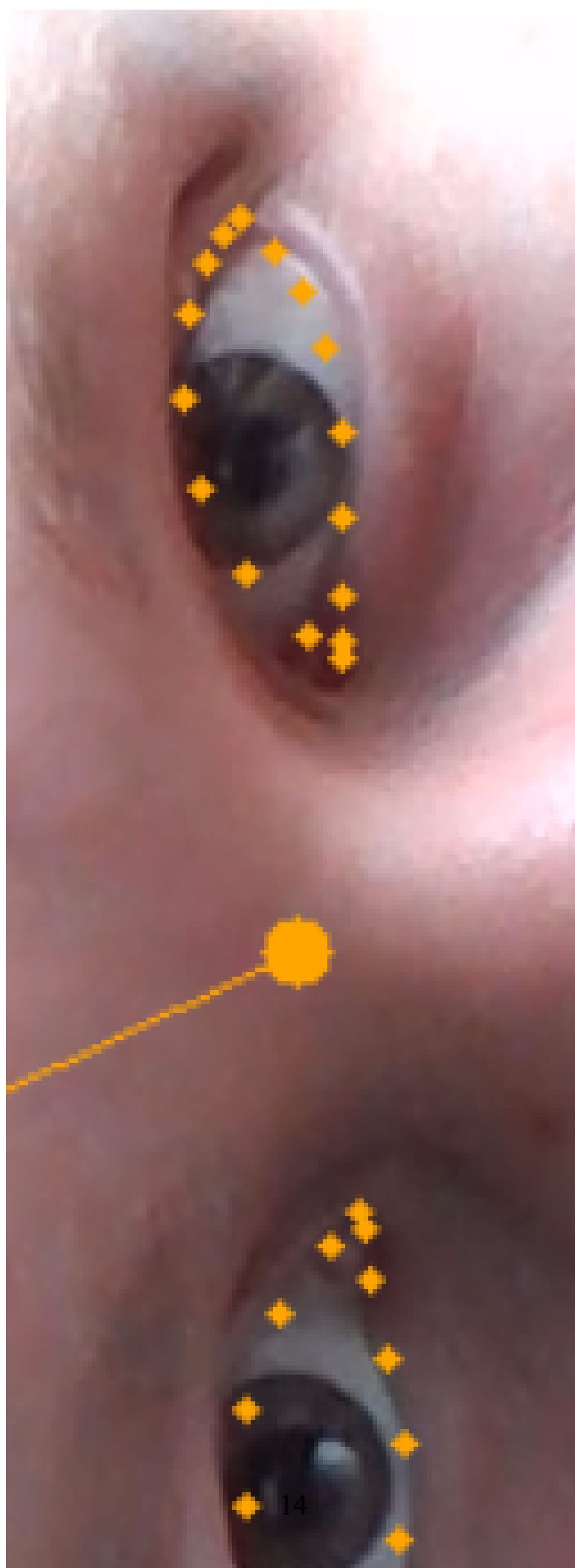
Для детекции лицевых ориентиров была выбрана нейросетевая модель MediaPipe Face Mesh, разработанная компанией Google. Эта модель обеспечивает баланс между точностью и производительностью, что критически важно для работы в реальном времени. Модель детектирует 468 трехмерных точек на лице с точностью локализации до 1-2 пикселей при разрешении кадра 640×480.

Инициализация модели осуществляется со следующими параметрами:

- `max_num_faces=1` – ограничение на одно лицо в кадре для увеличения производительности
- `refine_landmarks=True` – уточнение ключевых точек глаз и губ для повышения точности
- `min_detection_confidence=0.8` – порог уверенности детекции, фильтрующий ложные срабатывания
- `min_tracking_confidence=0.8` – порог уверенности отслеживания между кадрами
- `static_image_mode=False` – оптимизация для видеопотока с сохранением состояния между кадрами

Для каждого кадра выполняется следующий конвейер обработки:

1. Преобразование кадра из формата BGR (OpenCV) в RGB (MediaPipe)
2. Передача кадра в модель Face Mesh и получение массива нормализованных координат (от 0 до 1)
3. Выделение подмножества точек, соответствующих глазам: 16 точек для левого глаза (индексы 33, 7, 163, 144, 145, 153, 154, 155, 133, 173, 157, 158, 159, 160, 161, 246) и 16 точек для правого глаза (индексы 362, 382, 381, 380, 374, 373, 390, 249, 263, 466, 388, 387, 386, 385, 384, 398)
4. Масштабирование нормализованных координат до абсолютных значений в пикселях с учетом разрешения кадра



Определение открытости глаз и фильтрация невалидных кадров

Для обеспечения качества данных критически важна фильтрация кадров с закрытыми глазами, которые не содержат полезной энтропии. Алгоритм определения открытости глаз основан на расчете отношения вертикального и горизонтального размеров глазной щели.

Для каждого глаза вычисляются две метрики:

- **Вертикальное расстояние** между верхней и нижней точками века:

$$d_v^{left} = |y_{159} - y_{145}|, \quad d_v^{right} = |y_{386} - y_{374}|$$

где y_{159} и y_{145} — у-координаты верхней и нижней точек левого века, y_{386} и y_{374} — аналогичные точки правого глаза.

- **Горизонтальное расстояние** между внешним и внутренним углами глаза:

$$d_h^{left} = |x_{33} - x_{133}|, \quad d_h^{right} = |x_{362} - x_{263}|$$

Отношение открытости вычисляется по формуле:

$$O_{eye} = \min \left(1.0, \frac{d_v}{d_h} \times 2.0 \right)$$

Порог открытости установлен на уровне 0.4. Глаз считается открытым, если $O_{eye} > 0.4$. Для дополнительной проверки вычисляется симметрия между глазами:

$$S = 1.0 - |O_{left} - O_{right}|$$

Кадр считается валидным только если:

1. Оба глаза открыты ($O_{left} > 0.4$ и $O_{right} > 0.4$)
2. Симметрия между глазами превышает 0.85 ($S > 0.85$)
3. Разница в открытости не превышает 0.15 ($|O_{left} - O_{right}| < 0.15$)

Эти критерии позволяют отфильтровывать кадры с морганием, прищуриванием и асимметричным закрытием глаз, которые могут вносить систематические ошибки в измерения.

Детекция значимых движений взгляда

Детекция движений осуществляется по принципу сравнения текущей позиции взгляда с предыдущей значимой позицией. Алгоритм включает несколько этапов фильтрации для исключения микродвижений и дрейфа:

1. Вычисление положения взгляда: Позиция взгляда вычисляется комбинированным методом:

- **Метод центров глаз:** вычисляется среднее арифметическое центров левого и правого глаза

$$C_x = \frac{x_{left_center} + x_{right_center}}{2}, \quad C_y = \frac{y_{left_center} + y_{right_center}}{2}$$

- **Метод смещения зрачков:** оценивается смещение радужки относительно центра глаза

$$\Delta_x = \frac{(x_{468} - x_{left_center}) + (x_{473} - x_{right_center})}{2}$$

$$\Delta_y = \frac{(y_{468} - y_{left_center}) + (y_{473} - y_{right_center})}{2}$$

- **Комбинированная оценка:** итоговые координаты с весовыми коэффициентами

$$G_x = C_x + 0.3 \times \Delta_x, \quad G_y = C_y + 0.3 \times \Delta_y$$

2. Расчет параметров движения: Для каждой новой позиции взгляда вычисляются:

- **Величина движения:** евклидово расстояние от предыдущей значимой позиции

$$M = \sqrt{(G_x^{current} - G_x^{previous})^2 + (G_y^{current} - G_y^{previous})^2}$$

- **Направление движения:** угол в радианах относительно горизонтальной оси

$$\theta = \text{atan2}(G_y^{current} - G_y^{previous}, G_x^{current} - G_x^{previous})$$

3. Пороговая фильтрация: Движение считается значимым и регистрируется в системе, если выполняются условия:

- Величина движения превышает порог $M > 15$ пикселей (настроечный параметр)

- Время с последнего зарегистрированного движения превышает минимальный интервал $\Delta t > 0.05$ секунды
- Глаза открыты (проверка по алгоритму из предыдущего раздела)

Порог в 15 пикселей был определен эмпирически в ходе предварительных экспериментов как оптимальный баланс между чувствительностью (регистрация настоящих движений) и специфичностью (игнорирование шума и микродвижений).

2.4 Алгоритм извлечения энтропии

Источники энтропии

Система извлекает энтропию из шести независимых источников, каждый из которых вносит свой вклад в общую случайность генерируемой последовательности:

1. Пространственные координаты (X, Y):

- *Физический источник*: абсолютные координаты точки взгляда на экране
- *Разрядность*: 32-битные числа с плавающей запятой
- *Извлекаемые биты*: 4 младших бита из каждого байта координаты после умножения на 1000 и приведения к целому типу
- *Обоснование*: младшие биты координат наиболее чувствительны к микродвижениям и шуму измерений

2. Величина движения:

- *Физический источник*: евклидово расстояние между последовательными позициями взгляда
- *Разрядность*: 32-битное число с плавающей запятой
- *Извлекаемые биты*: 4 младших бита после умножения на 100 и приведения к целому типу
- *Обоснование*: величина движения отражает амплитуду саккад, которая варьируется непредсказуемо

3. Направление движения:

- *Физический источник*: угол движения в радианах (от $-\pi$ до π)
- *Преобразование*: квантование в 8 секторов: $\theta_{quantized} = \left\lfloor \frac{\theta + \pi}{2\pi} \times 8 \right\rfloor$
- *Извлекаемые биты*: 3 бита, представляющие номер сектора

- *Обоснование:* направление саккад зависит от визуальных стимулов и когнитивных процессов

4. Временные метки:

- *Физический источник:* системное время в микросекундах
- *Извлекаемые биты:* 4 младших бита микросекундной части времени

$$\text{bits} = \text{format}((\text{timestamp} - \lfloor \text{timestamp} \rfloor) \times 10^6 \mod 16, '04b')$$

- *Обоснование:* задержки в нейромоторной системе создают случайные вариации временных интервалов

5. Физиологические параметры:

- **Открытость глаз:** среднее значение открытости левого и правого глаза

$$\text{bits}_{\text{openness}} = \text{format}(\lfloor O_{\text{avg}} \times 100 \rfloor \mod 8, '03b')$$

- **Уровень внимания:** комплексный показатель, вычисляемый как взвешенная сумма:

$$A = 0.4 \times O_{\text{avg}} + 0.3 \times S + 0.3 \times (1 - \min(1.0, \frac{F_{\text{still}}}{50}))$$

где S — симметрия открытости, F_{still} — количество последовательных статичных кадров

$$\text{bits}_{\text{attention}} = \text{format}(\lfloor A \times 100 \rfloor \mod 8, '03b')$$

Квантование и бит-экстракция

Процесс преобразования аналоговых значений в битовую последовательность включает три этапа:

1. Предобработка и нормализация:

- Масштабирование значений в заданные диапазоны
- Фильтрация выбросов с использованием скользящего медианного фильтра
- Вычитание тренда для устранения медленных дрейфов

2. Квантование:

- Преобразование непрерывных значений в дискретные уровни

- Использование нелинейного квантования для областей с высокой плотностью вероятности
- Для координат: $Q(x) = \lfloor x \times 1000 \rfloor \bmod 256$
- Для величин: $Q(m) = \lfloor m \times 100 \rfloor \bmod 16$

3. Бит-экстракция:

- Извлечение младших битов из квантованных значений
- Для каждого параметра извлекается фиксированное количество бит
- Конкатенация битов из всех источников в единый поток

Таблица 2.1 — Параметры извлечения энтропии из различных источников

Источник энтропии	Извлекаемые биты	Общий вклад	Частота обновления
Координата X	4 бита	16.7%	30 Гц
Координата Y	4 бита	16.7%	30 Гц
Величина движения	4 бита	16.7%	1–10 Гц
Направление движения	3 бита	12.5%	1–10 Гц
Временная метка	4 бита	16.7%	30 Гц
Открытость глаз	3 бита	12.5%	30 Гц
Уровень внимания	2 бита	8.3%	30 Гц
Итого	24 бита	100%	30 Гц

Формирование ключа

Алгоритм добавления:

1. Битовая последовательность разбивается на байты (по 8 бит)
2. Каждый байт преобразуется в целое число
3. Байт добавляется в конец очереди

Алгоритм извлечения:

1. Проверка достаточности энтропии: требуется не менее 20 событий движения
2. Извлечение байтов из очереди в порядке добавления
3. Применение функции сжатия для увеличения плотности энтропии
4. Формирование ключа заданной длины (128 или 256 бит)

Функция хэширования: Следующим этапом обработки накопленных данных является применение криптографической функции хеширования для преобразования массива сырых байтов в криптографически стойкую битовую последовательность фиксированной длины. В реализованной системе использу-

ется алгоритм SHA-256 (Secure Hash Algorithm 256-bit), соответствующий требованиям современных стандартов криптографической защиты.

Применение детерминированной хеш-функции к накопленному пулу энтропии позволяет решить несколько важных задач:

- обеспечение равномерного распределения битов в выходной последовательности даже при наличии статистических аномалий во входных данных;
- устранение возможных корреляций между битами сырых измерений;
- достижение требуемого уровня криптографической стойкости за счёт однонаправленности и стойкости к коллизиям алгоритма SHA-256;
- формирование ключа фиксированной длины (256 бит), соответствующего стандартным требованиям для симметричного шифрования.

Таким образом, после накопления достаточного объёма энтропии массив сырых байтов подвергается хешированию, результатом которого является криптографический ключ, готовый к использованию в защищённых приложениях. Данный подход соответствует рекомендациям стандартов NIST и СТБ 34.101.27-2022 по постобработке данных от источников энтропии перед их использованием в качестве случайных чисел или ключевого материала.

2.5 Реализация

Язык Python и библиотеки

Программный комплекс реализован на языке Python 3.8+, что обеспечивает кроссплатформенность, богатую экосистему библиотек и относительно простую разработку прототипов. Выбор конкретных библиотек обусловлен их функциональностью и производительностью:

OpenCV (Open Source Computer Vision Library) версия 4.5+:

- Захват видеопотока с веб-камеры через интерфейс VideoCapture
- Преобразование цветовых пространств (BGR ↔ RGB)
- Визуализация результатов трекинга с рисованием графических примитивов
- Обработка изображений в реальном времени с оптимизацией под многоядерные процессоры

MediaPipe версия 0.8+:

- Предобученная модель Face Mesh для детекции 468 лицевых ориентиров
- АПИ для работы с нейросетевыми моделями в реальном времени

- Оптимизация для CPU с использованием TFLite
- Поддержка рефайнинга ключевых точек для повышения точности

NumPy версия 1.19+:

- Векторизованные вычисления для обработки массивов ключевых точек
- Статистические функции для анализа данных
- Эффективное хранение и манипуляции с многомерными массивами

Дополнительные библиотеки:

- `math` – математические функции (`atan2`, `sqrt`)
- `time` – работа с системным временем и задержками
- `collections` – структуры данных `deque` для эффективного управления историей
- `typing` – аннотации типов для улучшения читаемости кода

Логирование данных в бинарные файлы

Для обеспечения воспроизводимости экспериментов и последующего анализа реализована система логирования данных в нескольких форматах:

1. Бинарный формат (`newbits_data.bin`):

- *Назначение:* хранение сырых битовых последовательностей для тестирования энтропии
- *Структура:* поток байтов без заголовков, каждый байт соответствует 8 битам энтропии
- *Режим записи:* добавление в конец файла (`append binary mode`)

3. Криптографические ключи (`eye_crypto_key.bin`, `final_eye_key.bin`):

- *Назначение:* хранение сгенерированных ключей для использования в криптографических приложениях
- *Структура:* сырые байты ключа заданной длины (16 байт для 128 бит, 32 байта для 256 бит)
- *Дополнительно:* вывод HEX-представления в консоль для быстрой проверки

Интерфейс управления

Интерактивный интерфейс управления реализован через обработку клавиатурных событий в окне OpenCV. Это позволяет пользователю контролировать процесс сбора данных и генерации ключей без остановки программы:

Клавиша Q или ESC:

- *Действие:* корректное завершение программы
- *Обработка:* выход из основного цикла, освобождение ресурсов камеры, закрытие окон
- *Дополнительно:* автоматическая генерация финального ключа если накоплено более 50 движений

Клавиша S (Statistics):

- *Действие:* вывод текущей статистики в консоль
- *Отображаемые метрики:* общее количество обработанных кадров, количество зарегистрированных движений, процент движения, качество данных, средняя величина движения

Клавиша B (Bits):

- *Действие:* генерация 128-битной случайной последовательности
- *Процесс:* проверка достаточности энтропии, извлечение битов из пула, вычисление баланса, сохранение в файл
- *Выходные данные:* статистика баланса и путь к сохраненному файлу

Клавиша K (Key):

- *Действие:* генерация 256-битного криптографического ключа
- *Процесс:* проверка достаточности энтропии, извлечение 256 бит из пула, преобразование в байты, сохранение в бинарный файл
- *Выходные данные:* размер ключа в байтах, HEX-префикс, статистика баланса

Клавиша R (Reset):

- *Действие:* сброс всех накопленных данных
- *Очищаемые структуры:* история движений, история взгляда, история состояния глаз, счетчики статичных кадров и времени
- *Применение:* позволяет начать новый сеанс сбора данных без перезапуска программы

Глава 3

МЕТОДОЛОГИЯ ОЦЕНКИ КАЧЕСТВА ЭНТРОПИИ

3.1 Тестовый стенд

Для комплексной оценки качества энтропии, извлекаемой из движений глаз, был разработан специализированный тестовый стенд, позволяющий проводить измерения в различных сценариях активности пользователя. Целью создания тестового стенда являлось исследование влияния когнитивного состояния и типа деятельности на статистические характеристики генерируемой последовательности.

Сценарии сбора данных:

1. Случайные движения глазами:

- *Описание:* Пользователь произвольно перемещает взгляд по экрану без конкретной цели или визуальных стимулов.
- *Цель:* Оценка максимального потенциала энтропии при осознанной попытке генерации случайных движений.
- *Продолжительность:* 10 минут непрерывной активности.

2. Серфинг в интернете:

- *Описание:* Естественная деятельность пользователя при просмотре веб-страниц, чтении статей, переходе по ссылкам.
- *Цель:* Исследование качества энтропии в условиях реальной компьютерной деятельности.
- *Продолжительность:* 15 минут работы с браузером.

3. Просмотр фильма:

- *Описание:* Пользователь смотрит 10-минутный фрагмент динамичного фильма с активным сюжетом.
- *Цель:* Анализ энтропии при пассивном восприятии визуального контента.
- *Продолжительность:* 10 минут просмотра.

3.2 Инструменты оценки

Для оценки качества энтропии использовался стандарт NIST SP 800-90B “Recommendation for the Entropy Sources Used for Random Bit Generation”. Этот стандарт разработан Национальным институтом стандартов и технологий США (NIST) специально для тестирования источников энтропии, используемых в криптографических генераторах случайных чисел.

Тестовый пакет NIST SP 800-90B включает две основные утилиты:

- **ea_iid** (Entropy Assessment for IID data) – для тестирования независимых и одинаково распределённых данных
- **ea_non_iid** (Entropy Assessment for non-IID data) – для тестирования данных, которые могут не быть независимыми и одинаково распределёнными

Основные проверяемые метрики:

1. **H_{original} (энтропия на байт):** Мера неопределённости, содержащейся в одном байте последовательности

$$H_{original} = - \sum_{i=0}^{255} p_i \log_2 p_i$$

где p_i – вероятность появления байта со значением i

2. **H_{bitstring} (энтропия на бит):** Мера неопределённости, содержащейся в одном бите последовательности

$$H_{bit} = -[p \log_2 p + (1 - p) \log_2 (1 - p)]$$

где p – вероятность появления '1' в данной позиции

3. **min(H_{original}, 8 × H_{bitstring}):** Консервативная оценка минимальной энтропии, используемая для non-IID данных

$$H_{min} = \min(H_{original}, 8 \times H_{bitstring})$$

4. **Хи-квадрат тест (χ^2 test):** Проверка гипотезы о равномерном распределении байтов

$$\chi^2 = \sum_{i=0}^{255} \frac{(O_i - E_i)^2}{E_i}$$

где O_i – наблюдаемая частота, E_i — ожидаемая частота

5. **Тест на самую длинную повторяющуюся подстроку:** Обнаружение повторяющихся паттернов в последовательности
6. **Permutation tests:** Проверка гипотезы о независимости и одинаковом распределении (IID) данных

3.3 Критерии успешности

Для признания биологического генератора случайных чисел на основе движений глаз криптографически стойким, он должен удовлетворять следующим критериям:

1. Прохождение IID тестов:

- Хи-квадрат тест: $p\text{-value} > 0.001$
- Тест на самую длинную повторяющуюся подстроку: пройден
- Все permutation tests: пройдены
- Отсутствие значимых автокорреляций

2. Высокое значение $\min(H_{\text{original}}, 8 \times H_{\text{bitstring}})$:

- Для криптографических приложений: $H_{\min} > 7.0$ бит/байт
- Для генерации ключей: $H_{\min} \geq 6.5$ бит/байт
- Минимально допустимое: $H_{\min} > 5.0$ бит/байт

3. Устойчивость к non-IID атакам:

- Most Common Value Test: самый частый символ не должен встречаться значительно чаще ожидаемого
- Collision Test: не должно быть чрезмерного количества совпадений
- Markov Test: не должно быть значимых переходных вероятностей
- Все оценки энтропии в `ea_non_iid` должны давать схожие результаты

Глава 4

ЭКСПЕРИМЕНТАЛЬНЫЕ РЕЗУЛЬТАТЫ И ИХ АНАЛИЗ

4.1 Результаты тестирования

Результаты тестов для каждого сценария сведены в таблицу.

Таблица 4.1 — Результаты тестирования энтропии для IID-данных

Параметр	Серфинг	Случайные движения	Фильм
Объём данных (движения)	532	1130	330
Объём данных (байты)	1929	3457	1601
H_{original} (бит/байт)	6.388737	6.643665	6.080165
$H_{\text{bitstring}}$ (бит/бит)	0.965584	0.947404	0.918429
$\min(H_{\text{original}}, 8 \times H_{\text{bitstring}})$ (бит/байт)	6.388737	6.643665	6.080165
Хи-квадрат тест	Пройден	Не пройден	Пройден
Тест на самую длинную подстроку	Пройден	Пройден	Пройден
Permutation tests	Пройдены	Пройдены	Пройдены

Таблица 4.2 — Результаты тестирования энтропии для non-IID-данных

Параметр	Серфинг	Случайные движения	Фильм
Объём данных (движения)	532	1130	330
Объём данных (байты)	1929	3457	1601
H_{original} (бит/байт)	5.975382	6.643665	5.775921
$H_{\text{bitstring}}$ (бит/бит)	0.664687	0.699500	0.705646
$\min(H_{\text{original}}, 8 \times H_{\text{bitstring}})$ (бит/байт)	5.317494	5.596000	5.645168

4.2 Анализ результатов по сценариям

Серфинг в интернете

Ключевые результаты:

- H_{original} (IID): 5.975382 бит/байт
- Консервативная оценка (non-IID): 5.317494 бит/байт
- Все IID тесты пройдены

Анализ: Значение $H_{\text{original}} = 5.975382$ бит/байт указывает на высокое качество энтропии. Факт прохождения всех IID тестов свидетельствует о том, что данные обладают свойствами независимости и одинакового распределения. Консервативная оценка 5.317494 бит/байт учитывает возможные зависимости между битами и всё равно показывает хороший результат.

Случайные движения глазами

Ключевые результаты:

- H_{original} (IID): 6.643665 бит/байт (наивысшее значение)

- Консервативная оценка (non-IID): 5.596000 бит/байт
- Хи-квадрат тест не пройден

Анализ: Сценарий показывает наивысшее значение $H_{\text{original}} = 6.643665$ бит/байт (83.05% от максимума), что объясняется большей вариативностью сознательных движений. Однако неудача в хи-квадрат тесте указывает на отклонение от равномерного распределения байтов, вероятно вызванное сознательным контролем движений и психологическими факторами.

Просмотр фильма

Ключевые результаты:

- H_{original} (IID): 6.080165 бит/байт
- Консервативная оценка (non-IID): 5.645168 бит/байт
- Все IID тесты пройдены

Анализ: Значение $H_{\text{original}} = 6.080165$ бит/байт (76% от максимума) является самым низким среди трёх сценариев, что объясняется более предсказуемыми паттернами движений при просмотре контента. Однако все IID тесты пройдены, что указывает на хорошие статистические свойства. Интересно, что non-IID оценка (5.645) оказалась выше, чем в сценарии серфинга.

4.3 Ограничения и перспективы

Ограничения исследования:

1. Объём данных: 1320-1680 байт вместо рекомендуемых NIST 1 000 000 семплов
2. Временные рамки: 10-15 минут на каждый эксперимент

Перспективные направления:

1. Комбинирование сценариев для увеличения энтропии
2. Долгосрочные исследования с большими объёмами данных
3. Улучшение алгоритмов с адаптивным квантованием
4. Сравнение с другими биометрическими источниками
5. Мобильная реализация для смартфонов и планшетов

ЗАКЛЮЧЕНИЕ

В ходе выполнения данной курсовой работы была исследована возможность создания биологического генератора случайных чисел на основе поведенческого фактора – движений глаз человека. Работа включала теоретическое исследование, разработку программного комплекса, экспериментальную оценку качества энтропии и анализ полученных результатов.

Основные достижения работы:

1. Разработан программный комплекс для сбора энтропии движений глаз, включающий:

- Модуль трекинга взгляда на основе MediaPipe Face Mesh с точностью локализации 468 лицевых ориентиров
- Алгоритм детекции значимых движений с пороговой фильтрацией (15 пикселей, 0.05 с)
- Систему извлечения энтропии из 6 независимых источников
- Интерактивный интерфейс управления с возможностью генерации ключей

2. Создан экспериментальный стенд для оценки качества энтропии в трёх сценариях активности.

3. Проведена комплексная оценка качества энтропии с использованием стандарта NIST SP 800-90B:

- Для сценария серфинга: $H_{\text{original}} = 6.389$ бит/байт, все IID тесты пройдены
- Для случайных движений: максимальная энтропия 6.644 бит/байт
- Для просмотра фильма: $H_{\text{original}} = 6.080$ бит/байт, все IID тесты пройдены

Ключевые научные и практические результаты:

1. Доказана принципиальная возможность использования движений глаз в качестве источника энтропии для криптографии.

2. Установлено влияние типа активности на качество энтропии:

- Случайные движения дают максимальное количество энтропии
- Серфинг в интернете обеспечивает оптимальный баланс
- Просмотр фильма даёт стабильную, хотя и менее обильную энтропию

3. Определены практические рекомендации по использованию технологии.

Теоретическая значимость работы:

- Расширены представления о биометрических источниках энтропии

- Исследовано влияние когнитивных процессов на качество энтропии
- Разработана классификация сценариев использования движений глаз

Практическая ценность работы:

- Создан работающий прототип биологического ГСЧ
- Определены оптимальные параметры сбора данных
- Установлены временные рамки генерации ключей
- Разработана система оценки качества энтропии

Общий вывод:

Разработанный биологический генератор случайных чисел на основе движений глаз демонстрирует высокий потенциал для практического применения в криптографии. Технология обеспечивает доступность, безопасность, удобство и эффективность.

Проведённое исследование подтверждает, что движения глаз являются перспективным источником энтропии, способным дополнять и усиливать физические генераторы случайных чисел. Дальнейшее развитие технологии позволит создать конкурентоспособный биологический ГСЧ для массового использования в системах защиты информации.

СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ

1. Агиевич, С. В. Криптографические методы / С. В. Агиевич // НИИ прикладных проблем математики и информатики БГУ. — URL: <https://apmi.bsu.by/assets/files/agievich/cm.pdf> (дата обращения: 12.11.2025).
2. NIST Special Publication 800-90B. Recommendation for the Entropy Sources Used for Random Bit Generation // National Institute of Standards and Technology. — URL: <https://csrc.nist.gov/publications/detail/sp/800-90b/final> (дата обращения: 16.10.2025).
3. Lugaresi, C. MediaPipe: A Framework for Building Perception Pipelines / C. Lugaresi, J. Tang, H. Nash [et al.] // arXiv. — URL: <https://arxiv.org/abs/1906.08172> (дата обращения: 07.12.2025).
4. Ямпольский, В. В. Биометрические методы и средства идентификации личности / В. В. Ямпольский, Д. В. Михеев. — М.: Горячая линия–Телеком, 2011. — 312 с.

ПРИЛОЖЕНИЕ А

Исходный код программы

Листинг 4.1: Файл: EyeTracker.py (основной модуль)

```
1 import cv2
2 import numpy as np
3 import math
4 import time
5 import json
6 from collections import deque
7 from typing import List, Tuple, Optional
8 import mediapipe as mp
9
10 class AdvancedGazeTracker:
11
12     def __init__(self, screen_width: int = 1920, screen_height: int
13                 = 1080):
14         self.screen_width = screen_width
15         self.screen_height = screen_height
16
17         self.mp_face_mesh = mp.solutions.face_mesh
18         self.face_mesh = self.mp_face_mesh.FaceMesh(
19             max_num_faces=1,
20             refine_landmarks=True,
21             min_detection_confidence=0.8,
22             min_tracking_confidence=0.8,
23             static_image_mode=False
24         )
25
26         self.LEFT_EYE_INDICES = [33, 7, 163, 144, 145, 153, 154, 155,
27                                   133,
28                                   173, 157, 158, 159, 160, 161, 246]
29         self.RIGHT_EYE_INDICES = [362, 382, 381, 380, 374, 373, 390,
30                                   249,
31                                   263, 466, 388, 387, 386, 385, 384, 398]
32
33         # Параметры системы
34         self.movement_threshold = 15 # пикселей
35         self.min_movement_interval = 0.05 # секунд
```

```

34     self.eye_openness_threshold = 0.4
35
36     # Хранение данных
37     self.movement_events = deque(maxlen=1000)
38     self.gaze_history = deque(maxlen=100)
39     self.eye_state_history = deque(maxlen=50)
40
41     # Счетчики
42     self.frame_count = 0
43     self.last_movement_time = 0
44     self.consecutive_still_frames = 0
45     self.last_valid_gaze = None
46
47     print("Инициализация трекера взгляда завершена")
48
49     def process_frame(self, frame):
50         """Обработка одного кадра"""
51         rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
52         results = self.face_mesh.process(rgb_frame)
53
54         movement_detected = False
55         gaze_data = None
56
57         if results.multi_face_landmarks:
58             for face_landmarks in results.multi_face_landmarks:
59                 # Анализ состояния глаз
60                 eye_state = self._analyze_eye_state(face_landmarks)
61                 self.eye_state_history.append(eye_state)
62
63                 # Проверка открытости глаз
64                 if not eye_state['eyes_open']:
65                     self._draw_status(frame, "ГЛАЗА ЗАКРЫТЫ", (0, 0,
66                                     255))
67                     return frame, None
68
69                 # Вычисление направления взгляда
70                 current_gaze = self._calculate_gaze_position(
71                     face_landmarks, frame.shape)
72
73                 # Детекция движения
74                 movement_detected, movement_info = self.

```

```

73         _detect_movement(current_gaze)
74
75     if movement_detected:
76         # Формирование данных о движении
77         gaze_data = {
78             'x': float(current_gaze[0]),
79             'y': float(current_gaze[1]),
80             'timestamp': time.time(),
81             'movement_magnitude': movement_info['magnitude'],
82             'movement_direction': movement_info['direction'],
83             'eye_openness': eye_state['avg_openness'],
84             'attention_score': self.
85                 _calculate_attention_score(eye_state)
86         }
87
88         self.movement_events.append(gaze_data)
89         self.last_movement_time = time.time()
90         self.consecutive_still_frames = 0
91         self.last_valid_gaze = current_gaze
92     else:
93         self.consecutive_still_frames += 1
94
95     # Визуализация
96     self._draw_visualization(frame, face_landmarks,
97                             movement_detected, current_gaze)
98     status = "ДВИЖЕНИЕ" if movement_detected else f"ПОКОЙ
99         ({self.consecutive_still_frames})"
100     color = (0, 255, 0) if movement_detected else (0, 165,
101         255)
102     self._draw_status(frame, status, color)
103
104     self.frame_count += 1
105     return frame, gaze_data
106
107 def _analyze_eye_state(self, landmarks):
108     """Анализ состояния глаз"""
109     left_openness = self._calculate_eye_openness(landmarks, 'left
110         ')
111     right_openness = self._calculate_eye_openness(landmarks, '
112         right')

```

```

107     eyes_open = (left_openness > self.eye_openness_threshold and
108                 right_openness > self.eye_openness_threshold and
109                 abs(left_openness - right_openness) < 0.15)
110
111     return {
112         'eyes_open': eyes_open,
113         'left_openness': left_openness,
114         'right_openness': right_openness,
115         'avg_openness': (left_openness + right_openness) / 2,
116         'symmetry': 1.0 - abs(left_openness - right_openness)
117     }
118
119     def _calculate_eye_openness(self, landmarks, eye):
120         """Расчет открытости глаза"""
121         if eye == 'left':
122             vertical_points = [159, 145]
123             horizontal_points = [33, 133]
124         else:
125             vertical_points = [386, 374]
126             horizontal_points = [362, 263]
127
128         try:
129             # Вертикальное расстояние
130             top = landmarks.landmark[vertical_points[0]]
131             bottom = landmarks.landmark[vertical_points[1]]
132             vertical_dist = abs(top.y - bottom.y)
133
134             # Горизонтальное расстояние
135             left = landmarks.landmark[horizontal_points[0]]
136             right = landmarks.landmark[horizontal_points[1]]
137             horizontal_dist = abs(left.x - right.x)
138
139             if horizontal_dist == 0:
140                 return 0.0
141
142             openness = min(1.0, (vertical_dist / horizontal_dist) *
143                           2.0)
144             return openness
145         except:
146             return 0.0

```

```

147 def _calculate_gaze_position(self, landmarks, frame_shape):
148     """Вычисление позиции взгляда"""
149     h, w = frame_shape[:2]
150
151     # Центры глаз
152     left_center = self._calculate_eye_center(landmarks, self.
        LEFT_EYE_INDICES)
153     right_center = self._calculate_eye_center(landmarks, self.
        RIGHT_EYE_INDICES)
154
155     # Средняя точка между глазами
156     gaze_x = (left_center[0] + right_center[0]) / 2
157     gaze_y = (left_center[1] + right_center[1]) / 2
158
159     # Конвертация в координаты экрана
160     screen_x = gaze_x * self.screen_width
161     screen_y = gaze_y * self.screen_height
162
163     return (screen_x, screen_y)
164
165 def _calculate_eye_center(self, landmarks, indices):
166     """Вычисление центра глаза"""
167     points = []
168     for idx in indices:
169         point = landmarks.landmark[idx]
170         points.append((point.x, point.y))
171
172     x_coords = [p[0] for p in points]
173     y_coords = [p[1] for p in points]
174
175     center_x = sum(x_coords) / len(x_coords)
176     center_y = sum(y_coords) / len(y_coords)
177
178     return (center_x, center_y)
179
180 def _detect_movement(self, current_gaze):
181     """Детекция значимого движения"""
182     current_time = time.time()
183
184     # Проверка временного интервала
185     if current_time - self.last_movement_time < self.

```

```

min_movement_interval:
186     return False, {}
187
188 if self.last_valid_gaze is None:
189     self.last_valid_gaze = current_gaze
190     self.gaze_history.append(current_gaze)
191     return True, {'magnitude': 0, 'direction': 0}
192
193 # Вычисление величины движения
194 dx = current_gaze[0] - self.last_valid_gaze[0]
195 dy = current_gaze[1] - self.last_valid_gaze[1]
196 magnitude = math.sqrt(dx**2 + dy**2)
197
198 # Вычисление направления
199 direction = math.atan2(dy, dx)
200
201 # Проверка порога
202 if magnitude > self.movement_threshold:
203     self.gaze_history.append(current_gaze)
204     return True, {
205         'magnitude': magnitude,
206         'direction': direction
207     }
208
209     return False, {}
210
211 def _calculate_attention_score(self, eye_state):
212     """Расчет уровня внимания"""
213     openness_score = eye_state['avg_openness']
214     symmetry_score = eye_state['symmetry']
215     stability_score = 1.0 - min(1.0, self.
216         consecutive_still_frames / 50.0)
217
218     attention = (openness_score * 0.4 +
219         symmetry_score * 0.3 +
220         stability_score * 0.3)
221
222     return min(1.0, max(0.0, attention))
223
224 def _draw_visualization(self, frame, landmarks,
225     movement_detected, gaze_pos):

```

```

224     """Визуализация результатов"""
225     h, w = frame.shape[:2]
226
227     # Цветзависимостиотсостояния
228     color = (0, 255, 0) if movement_detected else (0, 165, 255)
229
230     # Отрисовкаключевыхточекглаз
231     all_eye_indices = self.LEFT_EYE_INDICES + self.
232         RIGHT_EYE_INDICES
233     for idx in all_eye_indices:
234         point = landmarks.landmark[idx]
235         x = int(point.x * w)
236         y = int(point.y * h)
237         cv2.circle(frame, (x, y), 2, color, -1)
238
239     # Отрисовкаточкивзгляда
240     gaze_x = int(gaze_pos[0] * w / self.screen_width)
241     gaze_y = int(gaze_pos[1] * h / self.screen_height)
242
243     if movement_detected:
244         cv2.circle(frame, (gaze_x, gaze_y), 10, (0, 0, 255), -1)
245         cv2.circle(frame, (gaze_x, gaze_y), 15, (255, 255, 255),
246             2)
247     else:
248         cv2.circle(frame, (gaze_x, gaze_y), 6, (0, 165, 255), -1)
249
250     # Линиякцентруэкрана
251     center_x, center_y = w // 2, h // 2
252     cv2.line(frame, (center_x, center_y), (gaze_x, gaze_y), color
253         , 1)
254
255     def _draw_status(self, frame, text, color):
256         """Отображение статуса"""
257         stats = self.get_statistics()
258
259         info_lines = [
260             f"Статус: {text}",
261             f"Движений: {stats['movement_count']}",
262             f"Статичных кадров: {self.consecutive_still_frames}",
263             f"Качество данных: {stats['data_quality']:.1f}%"
264         ]

```

```

262
263     for i, line in enumerate(info_lines):
264         y = 30 + i * 25
265         cv2.putText(frame, line, (10, y),
266                     cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 0, 0), 3)
267         cv2.putText(frame, line, (10, y),
268                     cv2.FONT_HERSHEY_SIMPLEX, 0.6, color, 1)
269
270     def get_statistics(self):
271         """Получение статистики"""
272         movement_count = len(self.movement_events)
273         total_frames = self.frame_count
274
275         if total_frames == 0:
276             return {
277                 'movement_count': 0,
278                 'movement_rate': 0.0,
279                 'data_quality': 0.0,
280                 'avg_magnitude': 0.0
281             }
282
283         # Средняя величина движения
284         magnitudes = [e['movement_magnitude'] for e in self.
285                       movement_events
286                       if 'movement_magnitude' in e]
287         avg_magnitude = np.mean(magnitudes) if magnitudes else 0.0
288
289         # Качество данных процент ( кадров открытыми глазами )
290         valid_eye_states = [s for s in self.eye_state_history if s['
291                             eyes_open']]
292         data_quality = (len(valid_eye_states) / len(self.
293                             eye_state_history)) * 100 \
294                             if self.eye_state_history else 0.0
295
296         return {
297             'movement_count': movement_count,
298             'movement_rate': (movement_count / total_frames) * 100,
299             'data_quality': data_quality,
300             'avg_magnitude': avg_magnitude,
301             'total_frames': total_frames
302         }

```



```

300
301
302 class EntropyGenerator:
303     """Генератор энтропии на основе движений глаз """
304
305     def __init__(self, gaze_tracker):
306         self.tracker = gaze_tracker
307         self.entropy_pool = deque(maxlen=1000)
308         self.bit_counter = 0
309
310     def extract_entropy_bits(self):
311         """Извлечение битов энтропии из движений """
312         if len(self.tracker.movement_events) < 5:
313             return []
314
315         bits = []
316         recent_movements = list(self.tracker.movement_events)[-10:]
317
318         for movement in recent_movements:
319             # Извлечение битов из координат
320             x_bits = self._extract_bits_from_float(movement['x'], 4)
321             y_bits = self._extract_bits_from_float(movement['y'], 4)
322
323             # Извлечение битов из величины движения
324             mag_bits = self._extract_bits_from_float(movement['
                 movement_magnitude'], 4)
325
326             # Извлечение битов из направления
327             dir_bits = self._extract_bits_from_direction(movement['
                 movement_direction'])
328
329             # Извлечение битов из временной метки
330             time_bits = self._extract_bits_from_timestamp(movement['
                 timestamp'])
331
332             # Извлечение битов из физиологических параметров
333             phys_bits = self._extract_physiological_bits(movement)
334
335             # Объединение всех битов
336             movement_bits = x_bits + y_bits + mag_bits + dir_bits +
                 time_bits + phys_bits

```

```

337         bits.extend(movement_bits)
338
339         # Добавление в пул энтропии
340         self._add_to_entropy_pool(movement_bits)
341
342     self.bit_counter += len(bits)
343     return bits
344
345     def _extract_bits_from_float(self, value: float, num_bits: int)
346     -> List[int]:
347         """Извлечение битов из числа плавающей запятой """
348         # Масштабирование и преобразование в целое число
349         int_value = int(abs(value) * 1000) % (1 << (num_bits * 2))
350         # Извлечение младших битов
351         bits = []
352         for i in range(num_bits):
353             bits.append((int_value >> i) & 1)
354         return bits
355
356     def _extract_bits_from_direction(self, direction: float) -> List
357     [int]:
358         """Извлечение битов из направления движения """
359         # Квантование направления в 8 секторов
360         sector = int((direction + math.pi) / (2 * math.pi) * 8) % 8
361         # Преобразование в 3 бита
362         bits = [(sector >> i) & 1 for i in range(3)]
363         return bits
364
365     def _extract_bits_from_timestamp(self, timestamp: float) -> List
366     [int]:
367         """Извлечение битов из временной метки """
368         # Извлечение микросекундной части
369         microseconds = int((timestamp - int(timestamp)) * 1e6)
370         # Извлечение 4 младших битов
371         bits = [(microseconds >> i) & 1 for i in range(4)]
372         return bits
373
374     def _extract_physiological_bits(self, movement: dict) -> List[
375     int]:
376         """Извлечение битов из физиологических параметров """
377         bits = []

```

```

374
375     # Открытость глаз (3 бита)
376     openness_int = int(movement['eye_openness'] * 100) % 8
377     bits.extend([(openness_int >> i) & 1 for i in range(3)])
378
379     # Уровень внимания (3 бита)
380     attention_int = int(movement['attention_score'] * 100) % 8
381     bits.extend([(attention_int >> i) & 1 for i in range(3)])
382
383     return bits
384
385 def _add_to_entropy_pool(self, bits: List[int]):
386     """Добавление битов в пул энтропии """
387     # Преобразование битов в байты
388     for i in range(0, len(bits), 8):
389         byte_bits = bits[i:i+8]
390         if len(byte_bits) == 8:
391             byte_value = 0
392             for j, bit in enumerate(byte_bits):
393                 byte_value |= (bit << j)
394             self.entropy_pool.append(byte_value)
395
396 def generate_random_key(self, key_size_bits: int = 256) ->
Optional[bytes]:
397     """Генерация криптографического ключа"""
398     if len(self.tracker.movement_events) < 20:
399         print(f"Недостаточно движений: {len(self.tracker.
400               movement_events)}/20")
401         return None
402
403     print(f"Генерация ключа {key_size_bits} бит...")
404
405     # Сбор энтропии
406     all_bits = []
407     while len(all_bits) < key_size_bits:
408         new_bits = self.extract_entropy_bits()
409         if new_bits:
410             all_bits.extend(new_bits)
411         else:
412             time.sleep(0.1)

```

```

413     # Обрезкадонужногоразмера
414     key_bits = all_bits[:key_size_bits]
415
416     # Преобразованиевбайты
417     key_bytes = bytearray()
418     for i in range(0, len(key_bits), 8):
419         byte_bits = key_bits[i:i+8]
420         if len(byte_bits) == 8:
421             byte_value = 0
422             for j, bit in enumerate(byte_bits):
423                 byte_value |= (bit << j)
424             key_bytes.append(byte_value)
425
426     # Проверкабаланса
427     ones = sum(key_bits)
428     proportion = ones / len(key_bits)
429     print(f"Сгенерирован ключ{len(key_bytes)} байт")
430     print(f"Баланс битов: {proportion:.3f} идеально( 0.5)")
431
432     return bytes(key_bytes)
433
434
435 def main():
436     """Основная функцияпрограммы"""
437     print("=" * 60)
438     print("БИОЛОГИЧЕСКИЙ ГЕНЕРАТОРСЛУЧАЙНЫХЧИСЕЛ")
439     print("На основеанализадвиженийглаз")
440     print("=" * 60)
441
442     # Инициализациятрекера
443     tracker = AdvancedGazeTracker()
444     entropy_gen = EntropyGenerator(tracker)
445
446     # Открытиевидеопотока
447     cap = cv2.VideoCapture(0)
448     if not cap.isOpened():
449         print("Ошибка: неудалосьоткрытькамеру")
450         return
451
452     print("\Управление:")
453     print(" S - Статистика")

```

```

454 print(" В - Генерация128 бит")
455 print(" К - Генерациябитного256- ключа")
456 print(" R - Сбросданных")
457 print(" Q/ESC - Выход")
458 print("\Начните движениеглазами...")
459
460 try:
461     while True:
462         ret, frame = cap.read()
463         if not ret:
464             break
465
466         # Обработкакадра
467         processed_frame, movement_data = tracker.process_frame(
468             frame)
469
470         # Отображениерезультата
471         cv2.imshow('Eye Tracker - BioRNG', processed_frame)
472
473         # Обработкаклавиш
474         key = cv2.waitKey(1) & 0xFF
475
476         if key == ord('q') or key == 27: # Q илиESC
477             print("\Завершение работы...")
478             break
479
480         elif key == ord('s'): # Статистика
481             stats = tracker.get_statistics()
482             print(f"\Статистика:")
483             print(f" Обработанокадров: {stats['total_frames']}")
484             print(f" Зарегистрировано движений: {stats['movement_count']}")
485             print(f" Процентдвижения: {stats['movement_rate']:.1f}%")
486             print(f" Качестводанных: {stats['data_quality']:.1f}%")
487             print(f" Средняявеличина движения: {stats['avg_magnitude']:.1f} пикс.")
488
489         elif key == ord('b'): # Генерация128 бит
490             print("\Генерацияn 128 случайныхбитов...")
491             bits = []

```

```

491     attempts = 0
492     while len(bits) < 128 and attempts < 100:
493         new_bits = entropy_gen.extract_entropy_bits()
494         if new_bits:
495             bits.extend(new_bits)
496             attempts += 1
497             time.sleep(0.05)
498
499     if len(bits) >= 128:
500         # Сохранение в файл
501         with open("random_bits.txt", "w") as f:
502             f.write(''.join(str(bit) for bit in bits[:128]))
503
504         ones = sum(bits[:128])
505         proportion = ones / 128
506         print(f"Сгенерировано 128 битов")
507         print(f"Баланс: {proportion:.3f} (0/1: {ones}/{128-ones})")
508         print(f"Сохранено в random_bits.txt")
509     else:
510         print(f"Не удалось собрать достаточно энтропии")
511
512 elif key == ord('k'): # Генерация ключа
513     key_bytes = entropy_gen.generate_random_key(256)
514     if key_bytes:
515         # Сохранение ключа
516         with open("crypto_key.bin", "wb") as f:
517             f.write(key_bytes)
518
519         # Вывод HEX представления
520         hex_key = key_bytes.hex()[:64] # Первые 32 байта
521         print(f"Ключ (HEX, первые 32 байта): {hex_key}")
522         print(f"Полный ключ сохранен в crypto_key.bin")
523
524 elif key == ord('r'): # Сброс
525     tracker.movement_events.clear()
526     tracker.gaze_history.clear()
527     tracker.eye_state_history.clear()
528     tracker.consecutive_still_frames = 0
529     print("\nДанные сброшены")
530

```

```

531 except KeyboardInterrupt:
532     print("\Прерванон пользователем")
533
534 finally:
535     # Освобождениересурсов
536     cap.release()
537     cv2.destroyAllWindows()
538
539     # Финальнаястатистика
540     stats = tracker.get_statistics()
541     print(f"\Итоговаяя статистика:")
542     print(f" Всегокадров: {stats['total_frames']}")
543     print(f" Всегодвижений: {stats['movement_count']}")
544     print(f" Собранобитовэнтропии: {entropy_gen.bit_counter}")
545
546
547 if __name__ == "__main__":
548     main()

```